

Greedy Algorithms

Design and Analysis of Algorithms

Brian C. Dean

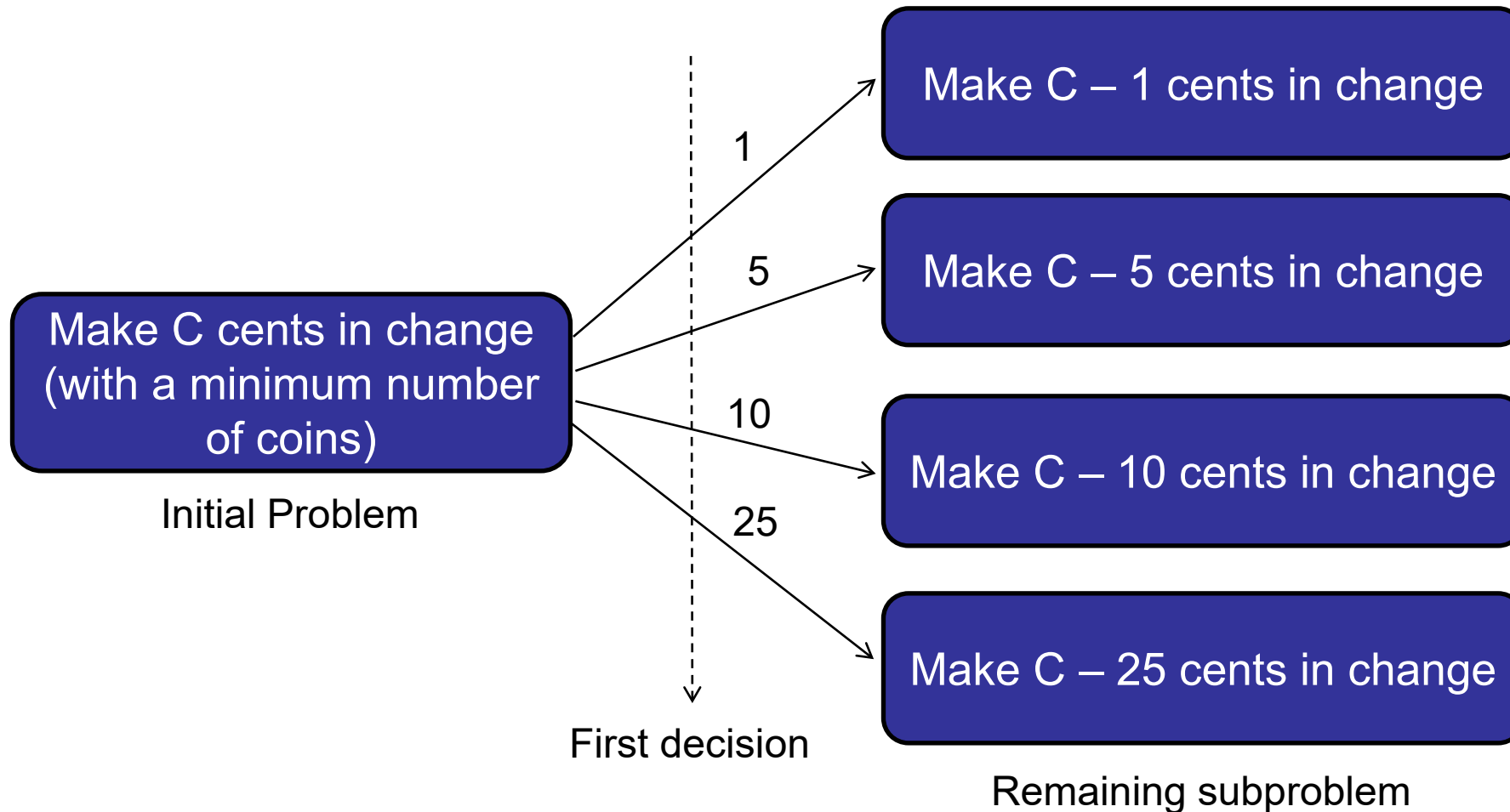
**School of Computing
Clemson University**



Making Change

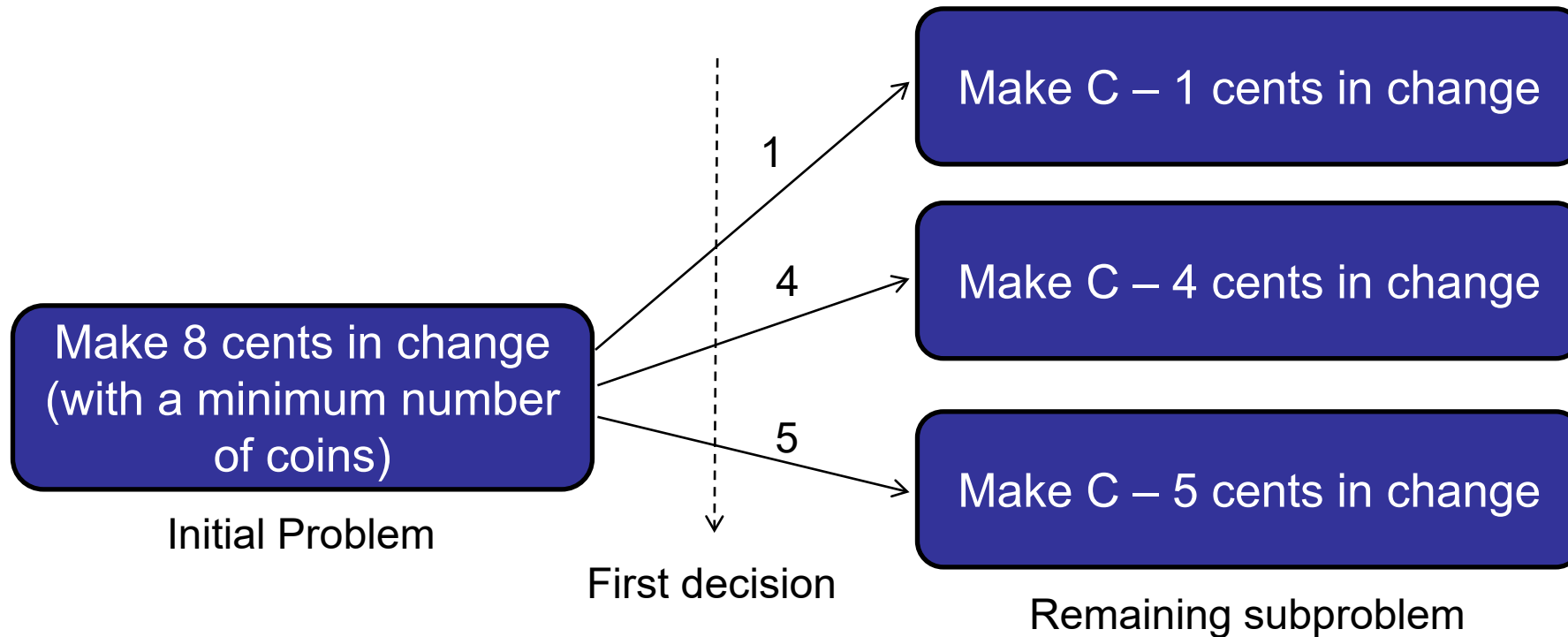
- We have N different denominations of coins (e.g., 1 cent, 5 cent, 10 cent, 25 cent).
- We can use as many coins of each denomination as we wish.
- What is the minimum number of coins we need in order to construct exactly C cents worth of change?

Sequential Decisions...



Greedy: Initial decision can be made safely and irrevocably made according to simple rule (e.g., use largest available coin).

Sequential Decisions...

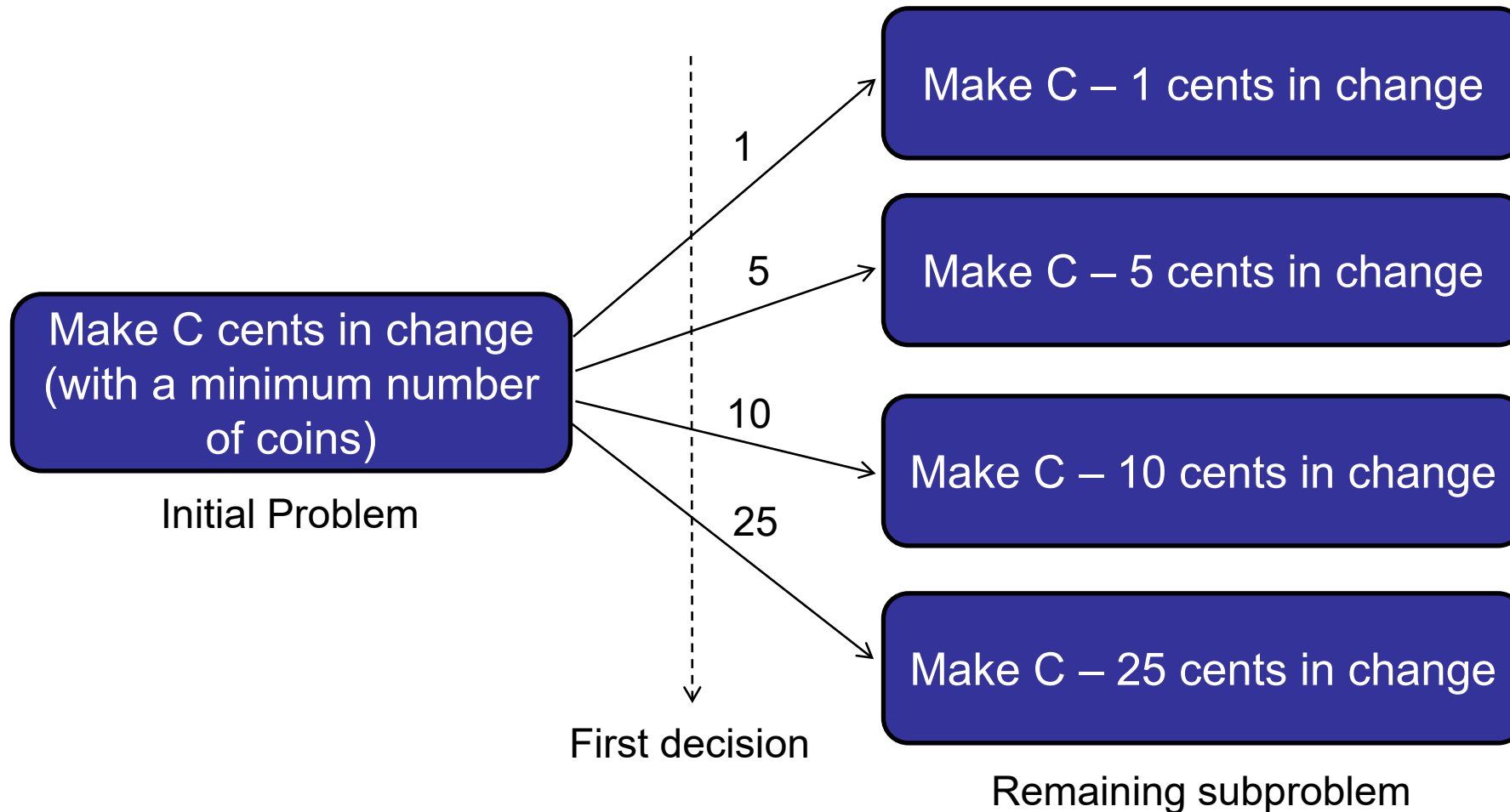


Unfortunately, greedy solutions may fail to produce an optimal solution due to their “short-sightedness”...

Optimal: 4, 4 (only 2 coins)

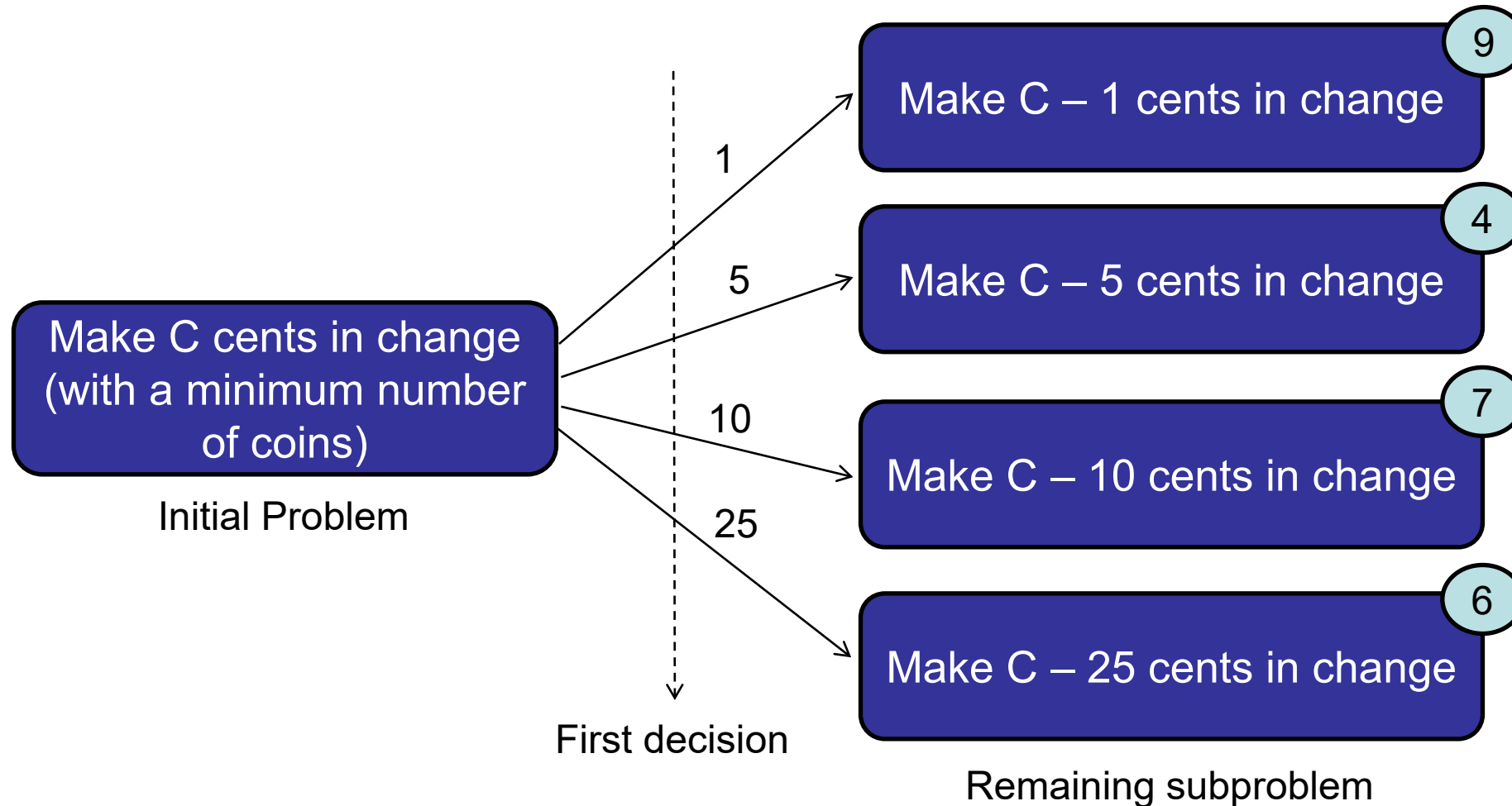
Greedy: 5, 1, 1, 1 (4 coins)

Sequential Decisions...



Exhaustive Search: Recursively try all possible first decisions, usually in some “greedy” order, usually with some sort of pruning heuristics to speed things up.

Sequential Decisions...



Dynamic Programming: Solve all possible subproblems from smallest to largest, making decisions easy!

Example: Activity Selection

- Given intervals $[a_i, b_i]$, select a disjoint subset of as many intervals as possible.
- When greedy algorithms work, something “natural” usually works as a greedy choice rule.
- E.g., first interval we pick might be:
 - The shortest interval
 - The interval overlapping the fewest other intervals
 - The interval starting earliest
 - ✓ The interval ending the earliest

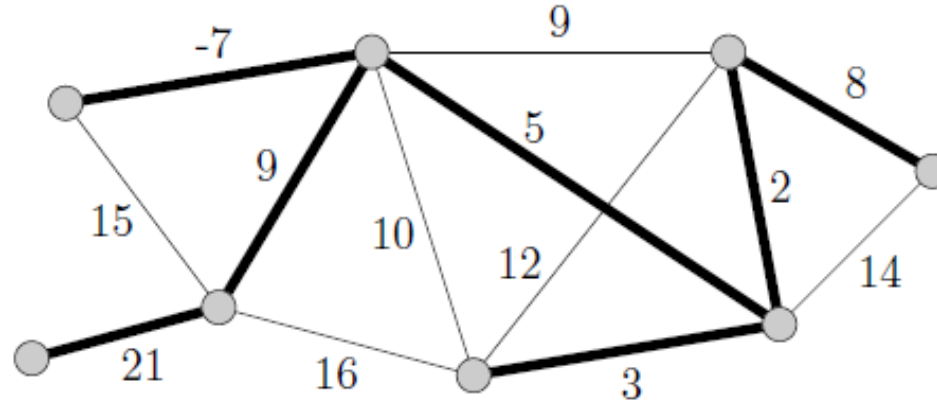
Example: Activity Selection

- Given intervals $[a_i, b_i]$, select a disjoint subset of as many intervals as possible.
- Greedy algorithms usually very fast (since they never look back) and simple to implement (often just involving sorting).
- Challenge is usually proving they give an optimal solution.
(often they don't, but nonetheless they still produce reasonably good solutions that are perfectly usable in practice)

Proving Optimality

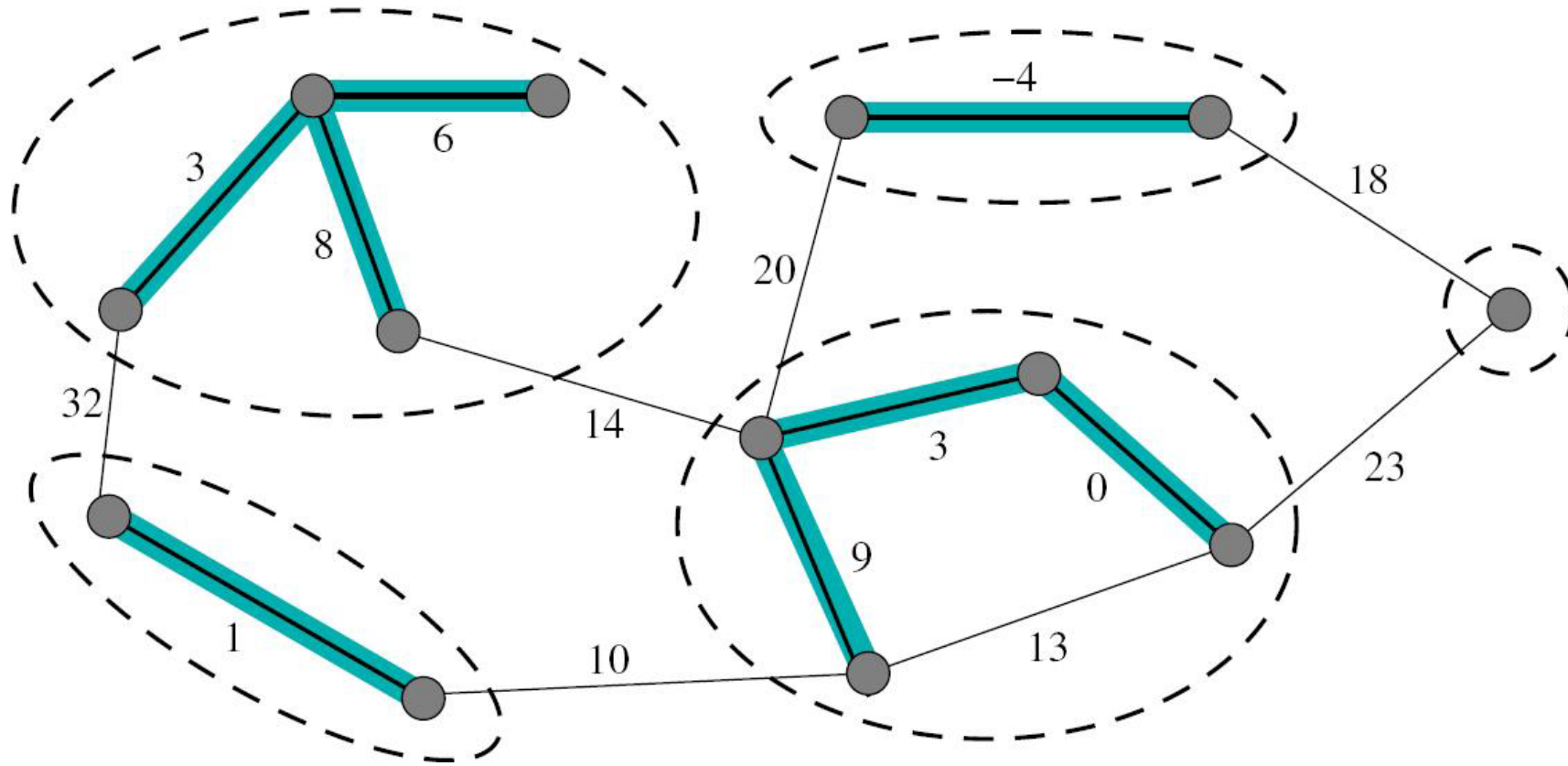
- Given intervals $[a_i, b_i]$, select a disjoint subset of as many intervals as possible.
- For purposes of contradiction, suppose the greedy solution isn't optimal...
- Now look at a hypothetical “optimal” solution.
- Somewhere, it made a different choice than the greedy solution. Argue that the greedy choice would have been smarter, leading to a better solution (contradicting our assumption of optimality or that greedy wasn't optimal).

Example #1: Kruskal's Minimum Spanning Tree Algorithm



- **Kruskal's algorithm** is perhaps the best known algorithm for computing an minimum spanning tree.
- It's a simple greedy algorithm:
 - Sort the edges in our graph in increasing order of cost.
 - For each edge in this sorted order:
Add it to our spanning tree if it doesn't create a cycle with edges we've already added (otherwise, skip it).

Example #1: Kruskal's Minimum Spanning Tree Algorithm



Snapshot in the middle of the algorithm

Example #2: Scheduling

- Given a set of jobs with:
 - Weights $w_1 \dots w_n$
 - Processing times $p_1 \dots p_n$
- Order them to minimize the weighted sum of their completion times: minimize $\sum w_j C_j$
- Special cases:
 - All weights 1 $\rightarrow$ order by increasing processing time.
 - All processing times 1 $\rightarrow$ order by decreasing weight.
- Greedy: ordering so that $w_1/p_1 \geq w_2/p_2 \geq \dots \geq w_n/p_n$